

Reviewer Pack — Algebraic Shifting Edge Count Bounds Monotone DNF Size for k...

Entry b106cd299f7b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-13 09:59:06 UTC

Algebraic Shifting Edge Count Bounds Monotone DNF Size for k-CLIQUE

Entry ID: b106cd299f7b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-13 09:59:06 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Shifting (Combinatorial Commutative Algebra) **Field B** (complexity object): Monotone DNF Formula Size

Statement:

For any monotone DNF formula Φ with m clauses over n variables, the number of edges in its algebraically shifted hypergraph satisfies $|E_{\text{shifted}}| \leq \log n + 2\sqrt{m}$. For the k-CLIQUE indicator formula, $|E_{\text{shifted}}| \geq n^{\{1/2\}} - 2\sqrt{k}$.

Rationale (proposer's reasoning):

Algebraic shifting preserves the hypergraph's face count while simplifying its structure. By measuring shifted edge counts, we expose trade-offs between clause complexity and formula size, with k-CLIQUE's shifted hypergraph exhibiting inherent geometric expansion.

Taxonomy category: MONOTONE_CLIQUE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7245db1e8e421481

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=1.5s

5.1 Generated Python source

```
import random
from itertools import combinations

def generate_dnf(n, m):
    clauses = []
    for _ in range(m):
        variables = random.sample(range(n), random.randint(1, n))
        clause = [(-1 if random.choice([True, False]) else 1) * var for var in variables]
        clauses.append(clause)
    return clauses

def lex_maximal_shifted_hypergraph(hypergraph):
    shifted_edges = set()
    for edge in hypergraph:
        for i in range(len(edge)):
            new_edge = tuple(sorted(edge[:i] + (edge[i] * -1,) + edge[i+1:]))
            if new_edge not in shifted_edges:
                shifted_edges.add(new_edge)
    return shifted_edges

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        m_max = n**2
        for _ in range(5): # Ensure at least 30 instances per seed
            m = random.randint(1, m_max)
            dnf = generate_dnf(n, m)

            hypergraph = set()
            for clause in dnf:
                hypergraph.update(combinations(range(n), len(clause)))

            shifted_edges = lex_maximal_shifted_hypergraph(hypergraph)
            num_edges = len(shifted_edges)
```

```

    if n == 2: # k-CLIQUE case
        k = m
        expected_min_edges = n**0.5 - 2 * (k**0.5)
        if num_edges < expected_min_edges:
            return {
                "metric_name": "E_shifted",
                "metric_value": num_edges,
                "instances_tested": len(n_values),
                "conjecture_holds": False,
                "counterexample": f"k-CLIQUE instance with n={n}, m={m} failed, expected a
            }
        else:
            expected_max_edges = 0
            for i in range(1, n+1):
                expected_max_edges += (i * (n - i))
            if num_edges > expected_max_edges + 2 * (m**0.5):
                return {
                    "metric_name": "E_shifted",
                    "metric_value": num_edges,
                    "instances_tested": len(n_values),
                    "conjecture_holds": False,
                    "counterexample": f"General DNF instance with n={n}, m={m} failed, expected
                }

    return {
        "metric_name": "E_shifted",
        "metric_value": sum(results) / len(results),
        "instances_tested": len(n_values),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30)) # Default to first 29 primes if

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    mean = sum(results) / len(results)
    std_dev = (sum((x - mean)**2 for x in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r <= log(len(seeds)) + 2 * (len(results)**0.5)) /

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean:.2f} std={std_dev:.2f} support_fraction={support_fra
    else:
        first_failing_seed = next(i for i, r in enumerate(results) if r > log(len(seeds)) + 2 * (
        print(f"RESULT: FALSIFIED counterexample=\"General DNF instance\" first_failing_seed={first

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
'}  
TRIAL: {'metric_name': 'E_shifted', 'metric_value': 75, 'instances_tested': 6, 'conjecture_holds': F  
TRIAL: {'metric_name': 'E_shifted', 'metric_value': 50, 'instances_tested': 6, 'conjecture_holds': F  
TRIAL: {'metric_name': 'E_shifted', 'metric_value': 25, 'instances_tested': 6, 'conjecture_holds': F  
TRIAL: {'metric_name': 'E_shifted', 'metric_value': 80, 'instances_tested': 6, 'conjecture_holds': F  
TRIAL: {'metric_name': 'E_shifted', 'metric_value': 80, 'instances_tested': 6, 'conjecture_holds': F  
TRIAL: {'metric_name': 'E_shifted', 'metric_value': 50, 'instances_tested': 6, 'conjecture_holds': F  
TRIAL: {'metric_name': 'E_shifted', 'metric_value': 80, 'instances_tested': 6, 'conjecture_holds': F  
TRIAL: {'metric_name': 'E_shifted', 'metric_value': 80, 'instances_tested': 6, 'conjecture_holds': F  
TRIAL: {'metric_name': 'E_shifted', 'metric_value': 80, 'instances_tested': 6, 'conjecture_holds': F  
TRIAL: {'metric_name': 'E_shifted', 'metric_value': 80, 'instances_tested': 6, 'conjecture_holds': F
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to undefined 'log' function, preventing reliable evaluation of support fraction | next: Fix the test code by importing math.log and re-run with proper error handling

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	51640
2	preregistration	ollama_remote	qwen3:8b	0	0	24073
3	novelty	ollama_remote	qwen3:8b	0	0	20538
4	novelty	ollama_remote	qwen3:8b	0	0	15917
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15701
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	34363
7	judge	ollama_remote	qwen3:8b	0	0	115654

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 277886 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/b106cd299f7b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b106cd299f7b.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/b106cd299f7b.tar.gz (if generated)